ProfiLLM: Utility-Aligned Agentic User Profiling for Industrial Ride-Hailing Dispatch [Scalable Data Science]

Tengfei Lyu*
HKUST(GZ)
Guangzhou, China
tlyu077@connect.hkust-gz.edu.cn

Zirui Yuan*
HKUST(GZ)
Guangzhou, China
zyuan779@connect.hkust-gz.edu.cn

Xu Liu
Didichuxing Co. Ltd
Beijing, China
leoliuxu@didiglobal.com

Kai Wan
Didichuxing Co. Ltd
Beijing, China
peterwan@didiglobal.com

Zihao Lu
Didichuxing Co. Ltd
Beijing, China
luzihao@didiglobal.com

Li Ma
Didichuxing Co. Ltd
Beijing, China
malimarey@didiglobal.com

Hao Liu†
HKUST(GZ)
Guangzhou, China
liuh@ust.hk

ABSTRACT

Bringing Large Language Models (LLMs) into industrial ride-hailing dispatch as semantic feature extractors over platform-scale behavioral logs is a compelling but under-explored data systems problem. Production matching pipelines remain dominated by structured numerical features, yet decisive behavioral signals, such as a driver's habitual aversion to certain regions, are inherently contextual and expressible as LLM-generated user profiles. However, scaling such profiling to a live, millisecond-latency dispatcher faces three intertwined constraints rarely addressed together: on a platform with millions of daily orders, logs exceed any LLM's context window by orders of magnitude; most users are long-tail, with too few interactions for per-user profiling; and surface-fluent profiles do not necessarily improve downstream prediction utility. We present ProfiLLM, an agentic LLM data pipeline that operationalizes utility-aligned user profiling for production matching systems through two modules. (1) Tool-Augmented Global Knowledge Mining equips an LLM agent with 27 analytical tools to mine platform-scale data, producing reusable global knowledge, adaptive user clustering rules, and region-level supply-demand priors. (2) Utility-Aligned Profile Exploration generates multiple candidate profiles per cluster, evaluates them via a lightweight downstream utility proxy, iteratively refines the best candidates and constructs preference pairs for DPO fine-tuning. Deployed at DiDi, ProfiLLM achieves up to +6.14% relative AUC gain and +4.35% GMV gain in simulation; a 14-day online A/B test yields +0.47% GMV and +0.33% completion.

PVLDB Reference Format:
Tengfei Lyu, Zirui Yuan, Xu Liu, Kai Wan, Zihao Lu, Li Ma, and Hao Liu. ProfiLLM: Utility-Aligned Agentic User Profiling for Industrial Ride-Hailing Dispatch [Scalable Data Science]. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

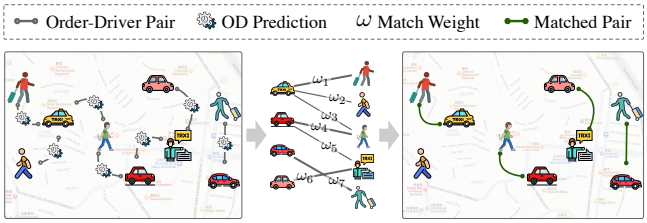
*Equal contribution. Work done during internship at Didichuxing Co. Ltd.
†Corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/ProfiLLM/ProfiLLM.

1 INTRODUCTION

Ride-hailing services have become an essential component of modern urban transportation, fundamentally changing how people commute and travel in cities worldwide. At the core of these platforms is order dispatching, which continuously matches passenger requests with available drivers under stringent real-time latency constraints [11, 32, 37]. To evaluate each potential match, the platform must anticipate the behavioral outcomes of each candidate order-driver (OD) pair along the fulfillment funnel: whether the driver will *accept* the order, and whether either party will *cancel* after acceptance. As illustrated in Figure 1, a typical production pipeline operates in three stages. (i) Predicting these per-stage outcomes for each candidate OD pair. (ii) Composing the predicted probabilities into matching weights that quantify the expected utility of each assignment. (iii) Computing a globally optimal assignment via bipartite matching, *e.g.,* Kuhn-Munkres [29]. Outcome prediction is the primary quality bottleneck among these stages. Its errors propagate directly into suboptimal matches, which increase passenger waiting time, reduce driver income, and degrade platform revenue.

Current production outcome predictors mainly rely on structured numerical features such as distance and price [18, 32, 36], leaving implicit semantic factors largely unexploited. However, decisive signals governing acceptance and cancellations are inherently contextual, such as a driver's long-term aversion to certain areas or a passenger's strict time-sensitivity during weekday mornings [10, 24]. They are often also conditional: as in the profile of Figure 3, a driver may accept low fares only for near-zero pickups and stop accepting late in a long shift. Such patterns are difficult to capture via handcrafted numerical features, but they can be naturally expressed as user profiles in language. They also differ across users in the same structured context. In City A, orders that share the fee quintile, pickup-ETA quintile, and time of day still show an average standard deviation of 24.9 percentage points in passengers' post-acceptance cancellation rates, as detailed in our online appendix. Large Language Models (LLMs) can distill complex behavioral trajectories into such contextual profiles, which motivates LLM-based user profiling for outcome prediction [14, 28, 31].



(a) Order-Driver Pair (b) Weight Assignment (c) Matched Results

Figure 1: Industrial order-dispatching pipeline: (a) outcome prediction for candidate order-driver (OD) pairs, (b) weight assignment, and (c) bipartite matching. Our work improves stage (a) via LLM-generated user profiles.

To validate this hypothesis, we conducted a pilot study on high-frequency users who have sufficient historical interactions. We employed an LLM to summarize their historical order trajectories into contextual profiles and incorporated it as additional features for outcome prediction. On average, the profile-enhanced model improves relative AUC by **3.71%** for driver cancellation and **9.64%** for passenger cancellation over the structured-only model. This suggests that LLM-generated profiles capture OD-relevant signals that structured features expose only partially.

However, the pilot study was conducted offline and restricted to high-frequency users with rich order histories. While these offline gains are encouraging, bridging the gap to production-scale online deployment introduces several fundamental obstacles. First, the majority of users are low-frequency and lack sufficient order history for reliable LLM-based profiling. Second, without platform-level knowledge to ground the generation, the LLM lacks a consistent reference frame. Conditioned on a single user's sparse trajectory, it produces profiles of highly variable quality. Third, even when order history is abundant, the generated profiles are not guaranteed to improve downstream prediction, as the LLM has no explicit signal to optimize for prediction utility. These observations motivate three interconnected challenges that must be addressed.

Challenge 1: Scalably mining global operational knowledge from massive historical data. Generating reliable knowledge requires understanding platform-level regularities such as temporal patterns, regional heterogeneity, and causal factors underlying order outcomes. Such global knowledge provides grounding context that individual user histories alone cannot supply. Yet raw logs are massive: a small city in our deployment yields 44.3M dispatching records over a 38-day window, far exceeding any LLM's context capacity, and a multi-city refresh is another order of magnitude larger. **Challenge 2: Adaptive user clustering under long-tail data distributions.** Even with global knowledge, per-user profiles remain infeasible for the dominant low-frequency population [9, 15, 23, 40]. Over the same 38-day window in City A, 96% of passengers appear in ≤ 30 orders, and the median passenger in only four. Although 73% of drivers are high-frequency, outcomes from both parties must be predicted for each candidate match, so passenger-side sparsity fundamentally limits user profiling. A principled clustering mechanism is required to group users by behavioral similarity so that each cluster accumulates sufficient data for reliable profile learning. **Challenge 3: Ensuring LLM-generated profiles are utility-aligned with downstream prediction.** With global knowledge and user clusters, LLMs can still produce fluent profile descriptions that fail to capture the specific factors driving behavioral decisions. As Table 2 later confirms, several strong off-the-shelf LLM backbones degrade downstream prediction AUC by up to -7.57% when used naively, i.e., with their profile embeddings fed into a retrained predictor. This shows that profile fluency is not a reliable proxy for prediction utility [17, 19]. A systematic mechanism is therefore needed to explore, evaluate, and refine profiles based on measurable downstream utility, while remaining compatible with strict online latency constraints.

To address these challenges, we introduce **ProfiLLM**, a practical data framework for deploying utility-aligned LLM user profiles in real-time ride-hailing matching. We devise a *Tool-Augmented Global Knowledge Mining* module in which an LLM agent, equipped with 27 analytical tools, analyzes platform-scale historical data following an Explore–Deepen–Validate–Synthesize paradigm. It produces (i) actionable global knowledge such as temporal patterns, weather impacts, and causal relationships, (ii) an adaptive user clustering at an appropriate granularity, and (iii) regional supply-demand priors from grid-level spatial analyses. We propose a *Utility-Aligned Profile Exploration* mechanism. For each user cluster, it generates candidate profiles conditioned on the mined global knowledge, scores them via rule-based prediction as a lightweight utility proxy, and iteratively refines the best candidates with feedback from prediction errors. The resulting utility comparisons yield preference pairs that drive DPO fine-tuning of a compact generator for cheap single-pass refresh. The generated profiles are embedded and added to the frozen production predictor through a gated fusion layer for real-time outcome prediction. Critically, all LLM inference occurs offline; the online system operates solely on pre-computed cluster-level profile embeddings, ensuring negligible added latency compatible with production requirements. Our contributions:

- We design **ProfiLLM**, an agentic LLM data pipeline that enforces a strict offline–online contract for behavioral profiling in industrial ride-hailing. All LLM reasoning is batch-offline, while the production dispatcher serves only pre-computed cluster-level profile embeddings. To our knowledge, this is the first deployed LLM-based user profiling pipeline for a production ride-hailing dispatcher [12].
- We design a **Tool-Augmented Global Knowledge Mining** module that exposes 27 analytical tools as a composable operator layer, driven by an LLM agent under an Explore-Deepen-Validate-Synthesize paradigm. The agent autonomously composes operator chains to extract actionable global knowledge, adaptive user-clustering rules, and regional supply-demand priors from platform-scale logs that exceed any LLM context window.
- We propose a **Utility-Aligned Profile Exploration** mechanism that turns prediction utility into a first-class optimization signal. For each user cluster, it generates multiple candidate profiles and ranks them by a lightweight LOGIC-rule proxy of downstream prediction. The resulting cross-candidate preferences drive DPO fine-tuning, which trains a compact generator toward downstream utility and enables cheap single-pass refresh.
- We **deploy ProfiLLM** on DiDi's production dispatcher. The online path adds sub-millisecond overhead with zero online

LLM inference. ProfiLLM yields up to **+6.14%** outcome-prediction AUC and **+4.35%** simulator GMV over pickup-distance matching. A 14-day City A switchback A/B test confirms gains of **+0.47%** GMV and **+0.33%** completion rate.

2 PROBLEM FORMULATION

Consider a ride-hailing platform operating in a city partitioned into a set of spatial grids $\mathcal{G} = \{g_1, g_2, \ldots, g_{|\mathcal{G}|}\}$. Let $\mathcal{P} = \{p_1, p_2, \ldots, p_M\}$ denote the set of passengers and $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$ denote the set of drivers. Each user $u \in \mathcal{P} \cup \mathcal{D}$ has a historical interaction sequence $\mathcal{H}_u = \{h_1^u, h_2^u, \ldots, h_{|\mathcal{H}_u|}^u\}$, where each record h_i^u contains order-level information such as timestamp, location, and trip details.

At each dispatching cycle, the system receives pending orders $\mathcal{O}$ and identifies candidate matches $\mathcal{C} \subseteq \mathcal{O} \times \mathcal{D}$. For each candidate pair $c = (o, d)$ with associated passenger p, the system predicts multiple order-level outcomes, such as order acceptance, that are used to compute matching weights. Let $\mathbf{y}_c = (y_c^{(1)}, y_c^{(2)}, \ldots, y_c^{(L)})$ denote the outcome vector, where L is the number of prediction tasks and each $y_c^{(l)} \in \{0, 1\}$ indicates whether the l-th stage in the order fulfillment process succeeds (*e.g.,* $l = 1$ for order acceptance, $l = 2$ for order completion). The prediction model estimates: $\hat{\mathbf{y}}_c = f(\mathbf{x}_c, \mathbf{e}_p, \mathbf{e}_d)$, where $\hat{\mathbf{y}}_c = (\hat{y}_c^{(1)}, \ldots, \hat{y}_c^{(L)})$ with $\hat{y}_c^{(l)} \in [0, 1]$ being the predicted probability for outcome l, $\mathbf{x}_c$ denotes structured features such as ETA, distance, and price, and $\mathbf{e}_p, \mathbf{e}_d$ are profile embeddings. These predictions are aggregated into a matching weight w_c quantifying the expected utility of each assignment. The optimal matching $\mathcal{M}^*$ that maximizes $\sum_{c \in \mathcal{M}} w_c$ is then computed via the Kuhn-Munkres algorithm, subject to one-to-one matching constraints. Our work improves outcome prediction by generating utility-aligned user profiles whose embeddings $\mathbf{e}_p$ and $\mathbf{e}_d$ complement structured features.

3 METHODOLOGY

3.1 System Overview

Production setting. ProfiLLM is designed and deployed in DiDi's production dispatching service. The dispatcher operates in fixed 2-second cycles; in each cycle, it enumerates candidate OD pairs, predicts per-pair behavioral outcomes, composes them into matching weights, and solves the assignment via the Kuhn–Munkres (KM) algorithm under a ∼200 ms end-to-end latency budget. Since LLM inference at per-OD-pair scale is incompatible with this budget, ProfiLLM materializes a strict offline–online decoupling as the three-layer pipeline in Figure 2.

The three layers. Offline, Layer 1 mines global knowledge $\mathcal{K}$, a user clustering $\mathcal{A}$, and regional supply–demand priors $\mathcal{R}$ with a tool-augmented LLM agent, and Layer 2 ranks candidate profiles of each cluster by a LOGIC-rule proxy of prediction utility, refines the best, embeds the five best into $\mathbf{e}_a$, and uses the resulting preferences for a one-time DPO fine-tune of a compact generator for refresh. Online, in Layer 3, the latency-critical dispatcher performs only two operations per OD pair: (i) a lookup of the user's cluster, which is assigned offline with $\mathcal{A}$, and (ii) a cached embedding fetch returning $\mathbf{e}_p$ and $\mathbf{e}_d$, together with the regional priors $\mathbf{r}_{g,s} \in \mathcal{R}$ of the pickup and destination grids. A small gated fusion layer adds the embeddings to the frozen production multi-task predictor, and the priors raise the matching weight where pickup demand exceeds supply or the destination offers good follow-up demand; KM assignment proceeds unchanged. *No LLM inference occurs on this path*, and combined overhead is under 0.01 ms per pair.

The offline–online contract. Layers 1–2 emit three artifacts that cross into Layer 3: the cluster-assignment rule set $\mathcal{A}$, a few KB of standardization parameters and cluster centroids; the cluster-embedding table $\{\mathbf{e}_a\}$ with the fusion weights; and the regional prior table $\mathcal{R}$. This narrow interface is the structural reason ProfiLLM operates within DiDi's existing latency budget without modifying the downstream matching components.

3.2 Tool-Augmented Global Knowledge Mining

Directly feeding massive historical logs to LLMs is infeasible due to context length limits and computational costs. To address Challenge 1, we design a tool-augmented knowledge mining module that enables an LLM agent to systematically analyze platform-scale data through structured tool invocations.

3.2.1 Tool Design and Categorization. We design a comprehensive toolkit $\mathcal{T}$ containing 27 analytical tools organized into six categories: statistical, causal, knowledge, validation, spatiotemporal, and contextual. Each tool $t_i \in \mathcal{T}$ is defined by a tuple ($name_i$, $desc_i$, $params_i$, $func_i$), where $name_i$ is the tool identifier, $desc_i$ provides a natural language description for the LLM to understand its functionality, $params_i$ specifies the required input parameters, and $func_i$ implements the actual computation logic. The tools are designed to be composable, allowing the agent to chain multiple tools to answer complex analytical questions. Each tool runs SQL and statistical routines next to the data and returns only aggregates, so the agent never reads raw records; the Explore phase uses AggregateStats, CompareSegments, DetectPeakPeriods, DayOfWeekPattern, and HourlyPattern, and our online appendix lists all tools by phase with their parameters.

3.2.2 Explore-Deepen-Validate-Synthesize Paradigm. We structure the knowledge mining process into four phases that guide the LLM agent through systematic data exploration: **(1) Explore:** The agent invokes basic statistical tools to compute platform-level metrics such as completion and acceptance rates, examine feature distributions, and surface preliminary patterns. **(2) Deepen:** Based on exploration findings, the agent conducts focused analyses on promising directions, including hourly cancellation variations, grid-level supply–demand imbalances, and user segmentation by historical order patterns. **(3) Validate:** Discovered patterns are subjected to statistical hypothesis testing and causal analysis; only findings with significant effect sizes and passing appropriate tests are retained. This filter is deliberately conservative: short-lived events such as a new road closure are left to the predictor's real-time features, such as live-traffic ETAs and minute-level supply–demand counts; detecting them within the agent is future work. **(4) Synthesize:** Validated findings are consolidated into three structured outputs: *Global Knowledge* $\mathcal{K}$, containing platform benchmarks, temporal regularities, weather impacts, and causal relationships; *User Clustering Rules* $\mathcal{A}$, a centroid-based clustering of standardized user-history statistics such as active hours and cancellation rates, computed by the UserClustering operator, whose clusters the agent labels

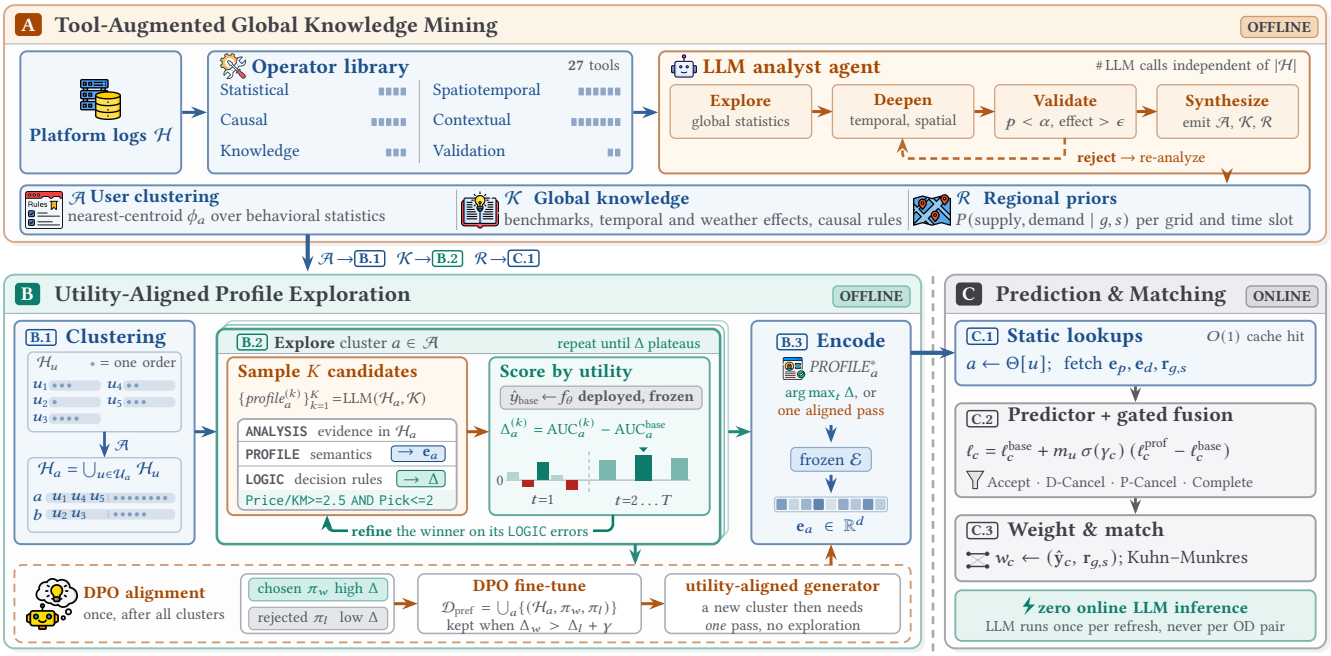


Figure 2: Overview of ProfiLLM. (a) An LLM analyst chains 27 operators over platform-scale logs into three reusable artifacts: clustering rules $\mathcal{A}$, global knowledge $\mathcal{K}$, and regional priors $\mathcal{R}$. (b) Per cluster, candidate profiles are ranked by the offline AUC gain Δ of their `LOGIC` rules alone; the winner is refined, and cross-cluster preferences train a one-time DPO-aligned generator. (c) Only Θ, $\{\mathbf{e}_a\}$, and $\mathcal{R}$ cross into serving, which reuses the production predictor and KM matcher with zero online LLM inference.

with short signatures; and *Regional Priors* $\mathcal{R}$, storing grid-level and time-slot-level supply-demand statistics $P(\textit{supply}, \textit{demand} \mid g, s)$ with derived metrics such as expected waiting time and fulfillment rate.

Bounded LLM context. The agent sees only aggregated tool outputs of 4.2–4.9K characters per call on average, truncated at 8K; a session over the 36.8M City A records of November 2025 issued 27–31 tool calls and took 3.4–4.3 minutes, including the final report.

3.3 Utility-Aligned Profile Exploration

With the mined global knowledge and clustering rules, we now address Challenges 2 and 3 through a cluster-level profile exploration mechanism. The key insight is that: (1) clustering aggregates sufficient historical data for reliable profiling even for low-frequency users, and (2) profile quality should be measured by downstream prediction performance rather than surface-level fluency.

3.3.1 User Clustering. Using the clustering rules $\mathcal{A}$ from the knowledge mining module, we partition all users into $|\mathcal{A}|$ clusters. For each user $u \in \mathcal{P} \cup \mathcal{D}$, we assign them to the most appropriate cluster based on their historical behavior: $a^*(u) = \arg\max_{a \in \mathcal{A}} \phi_a(\mathcal{H}_u)$, where ϕ_a is the compatibility of u's standardized behavioral statistics $\mathbf{z}_u$ with the centroid $\boldsymbol{\mu}_a$ of cluster a:

$$\phi_a(\mathcal{H}_u) = \frac{\exp(-\|\mathbf{z}_u - \boldsymbol{\mu}_a\|^2)}{\sum_{b \in \mathcal{A}} \exp(-\|\mathbf{z}_u - \boldsymbol{\mu}_b\|^2)} \in [0, 1], \tag{1}$$

so $a^*(u)$ is the nearest centroid, and cluster membership is the indicator $\mathbb{1}[a^*(u) = a]$. Users within the same cluster share similar behavioral patterns, allowing us to learn a shared user profile that captures cluster-level regularities.

Let $\mathcal{U}_a = \{u : a^*(u) = a\}$ denote the set of users assigned to cluster a, and $\mathcal{H}_a = \bigcup_{u \in \mathcal{U}_a} \mathcal{H}_u$ denote the aggregated historical orders for the cluster. This aggregation ensures sufficient data volume for reliable profile learning, addressing Challenge 2.

Default embedding. Members of clusters without enough labeled outcomes for the held-out evaluation of Section 3.3.3 and users unseen in training receive the zero embedding $\mathbf{e}_0$, for which the fusion layer returns exactly the production prediction. Users far from all centroids are still assigned to the nearest one.

3.3.2 Structured Profile Generation. For each cluster $a \in \mathcal{A}$, we prompt the LLM to generate K diverse candidate user profiles based on the aggregated cluster history $\mathcal{H}_a$ and global knowledge $\mathcal{K}$. Each candidate profile follows a structured three-part format:

$$\{profile_a^{(k)}\}_{k=1}^K = \text{LLM}(\mathcal{H}_a, \mathcal{K}, prompt_{gen}) \tag{2}$$

Each profile $profile_a^{(k)}$ consists of three components: (1) **ANALYSIS**: A detailed examination of the cluster's behavioral patterns observed in the historical data, identifying key factors that influence order outcomes. (2) **PROFILE**: A semantic description characterizing the cluster's order completion patterns, behavioral preferences, and service compatibility signals, such as a preference for shorter-pickup, higher-fare rides. (3) **LOGIC**: Executable decision rules derived from the analysis and profile, expressed as Boolean conditions over order features, for example `(Price_Per_KM >= 2.5) AND (Pick_KM <= 2.0)`. The prompt $prompt_{gen}$ instructs the LLM to generate diverse profiles emphasizing different aspects such as

PROFILE. A "Strategic Marathoner" who prefers high-total-fare orders to minimize pickups but caps the workday at 12 hours; may grab low-value orders instinctively but rarely completes them unless the passenger is essentially at the driver's location. Rules: (1) reject all orders once `Hours` > 12; (2) accept near-zero pickups (`Pick_Min` < 1) if `Price` ≥ 10 BRL; (3) prefer `Price` ≥ 16 if `Pick_Min` < 7; (4) reject `Price` 10–15 if `Pick_Min` > 3; (5) likely cancel `Price` < 10 unless the pickup is very convenient.

LOGIC_ACCEPT. `(Hours<12) and ((Pick_Min<1.0 and Price>=10) or (Price>=16 and Pick_Min<7.0))`

Figure 3: An abridged driver-cluster profile written by Gemini-3-Flash; more examples are in our online appendix.

temporal patterns, fare and trip preferences, and spatial preferences, while remaining grounded in the cluster's actual order history. Each prompt contains the benchmarks from $\mathcal{K}$, cluster summary statistics, and at most ten re-sampled recent records per outcome group, so prompts stay within 2.1–3.2K tokens regardless of cluster size. Figure 3 shows a generated profile.

3.3.3 Utility-Based Profile Evaluation. A critical design choice is how to evaluate profile quality efficiently. Rather than training full prediction models for each candidate, which would be computationally prohibitive, we leverage the LOGIC component as a lightweight proxy for utility evaluation. The intuition is that if the rules extracted from a profile accurately predict outcomes, then the profile captures meaningful behavioral patterns that are also likely to benefit the embedding-based predictor.

Specifically, for each candidate profile $profile_a^{(k)}$, we extract its LOGIC rules and apply them to the cluster's historical order-driver pairs $\{(c_i, y_i)\}_{i=1}^{n_a}$ to generate rule-based predictions $\{\hat{y}_i^{(k)}\}$:

$$\hat{y}_i^{(k)} = \texttt{evaluate}(LOGIC_a^{(k)}, c_i). \tag{3}$$

We then compute a fused prediction by blending the base model's output with the LOGIC-rule prediction. Specifically, for each order-driver pair c_i in cluster a and each prediction task l such as driver cancellation, the base production model produces a probability estimate $\hat{y}_{i,\text{base}}^{(l)} \in [0,1]$, while the LOGIC rules yield a binary prediction $\hat{y}_{i,\text{logic}}^{(k)} \in \{0,1\}$. The fused prediction is obtained via a convex combination controlled by a blending coefficient λ:

$$\hat{y}_{i,\text{fused}}^{(k)} = (1-\lambda)\cdot \hat{y}_{i,\text{base}}^{(l)} + \lambda \cdot \hat{y}_{i,\text{logic}}^{(k)} \tag{4}$$

where $\lambda \in [0,1]$ governs the relative influence of the LOGIC rules. The fused AUC is then computed against the labels $\{y_i^{(l)}\}_{i=1}^{n_a}$:

$$\text{AUC}_a^{(k)} = \text{ComputeAUC}\left(\{y_i^{(l)}\}_{i=1}^{n_a},\ \{\hat{y}_{i,\text{fused}}^{(k)}\}_{i=1}^{n_a}\right). \tag{5}$$

The utility gain of profile k is measured as the AUC improvement over the base production model alone: $\Delta_a^{(k)} = \text{AUC}_a^{(k)} - \text{AUC}_a^{\text{base}}$, where $\text{AUC}_a^{\text{base}} = \text{ComputeAUC}(\{y_i^{(l)}\},\ \{\hat{y}_{i,\text{base}}^{(l)}\})$ is AUC achieved by the base model without LOGIC-rule augmentation. A positive $\Delta_a^{(k)}$ indicates that the LOGIC rules encode complementary behavioral signals not captured by the production predictor, so the proxy favors profiles contributing genuine discriminative power beyond the existing model over superficially plausible descriptions that merely recapitulate what structured features already capture.

Data split and λ. Generation and selection use only the 38-day training window; the 5-day test window is never used. Per cluster, Δ is computed on a held-out selection split that includes the most recent records of each outcome, and the records shown in the prompt come from the disjoint context split. λ is chosen per candidate and task from $\{0.01, \ldots, 0.99\}$ by the mean AUC on both splits; in a post-hoc sweep, the best value is mostly 0.9, and 0.1 or 0.9 for driver cancellation. The online appendix gives the split sizes and the λ analysis.

3.3.4 Iterative Profile Refinement. We adopt an iterative refinement strategy to progressively improve profile quality. Starting from the best-performing initial candidate $profile_a^{(k^*)}$ where $k^* = \arg\max_k \Delta_a^{(k)}$, we prompt the LLM to generate refined versions that address identified weaknesses:

$$profile_a^{(t+1)} = \text{LLM}(profile_a^{(t)}, \mathcal{H}_a, feedback^{(t)}, prompt_{refine}) \tag{6}$$

where $feedback^{(t)}$ summarizes the prediction errors of the current profile's LOGIC rules, such as false positives and false negatives with their order features. The refinement continues for T iterations or until the utility gain plateaus, yielding the selected profile $profile_a^* = \arg\max_{t\in\{1,\ldots,T\}} \Delta_a^{(t)}$. The five candidates with the highest Δ_a are retained for per-user selection.

3.3.5 DPO Fine-tuning for Profile Generation. The exploration process naturally produces preference pairs that can be leveraged for DPO fine-tuning. For each cluster a, we construct preference pairs by comparing profiles with different utility gains:

$$\mathcal{D}_{\text{pref},a} = \{(\mathcal{H}_a, profile_w, profile_l) : \Delta_a^{(w)} > \Delta_a^{(l)} + \gamma\} \tag{7}$$

where γ is a margin threshold ensuring meaningful preference differences. Aggregating across all clusters yields a preference dataset $\mathcal{D}_{\text{pref}} = \bigcup_{a\in\mathcal{A}} \mathcal{D}_{\text{pref},a}$. We perform a one-time offline DPO fine-tuning of the LLM using the standard DPO objective [19] on $\mathcal{D}_{\text{pref}}$. This fine-tuning trains the generator toward downstream prediction utility, so that profiles for new or refreshed clusters are generated in a single pass without requiring additional exploration iterations.

All profiles describe cluster-level behavioral patterns such as temporal preferences rather than individual demographic attributes, so that the profiling mechanism operates on aggregate behavioral signals and limits individual-level privacy exposure. This matters even though profiles are internal: some of our offline LLM calls go to external APIs, and data-protection rules such as Brazil's LGPD require data minimization. Prompts carry no user identifiers or locations; this is data minimization, not a formal privacy guarantee.

3.4 Online Outcome Prediction and Matching

3.4.1 Profile Embedding and Caching. Each cluster's textual PROFILE component is converted into a dense embedding via a pre-trained text encoder $\mathcal{E}$: $\mathbf{e}_a = \mathcal{E}(PROFILE_a^*) \in \mathbb{R}^d$. For each user u, their profile embedding is simply the embedding of their assigned cluster: $\mathbf{e}_u = \mathbf{e}_{a^*(u)}$; for ProfiLLM, each user takes the one of its cluster's five retained candidates with the lowest fusion loss on the user's validation interactions, which are disjoint from the test window. Since profiles are at the cluster level, the number of embeddings to cache is bounded by $5|\mathcal{A}|$, making caching highly efficient. All cluster embeddings are pre-computed and stored in a distributed cache with sub-millisecond lookup latency.

3.4.2 Online Feature Integration and Matching. At serving time, for each candidate pair $c = (o, d)$ with passenger p, the frozen production multi-task predictor computes a logit t_c^{base} per task from the structured features $\mathbf{x}_c$. A lightweight fusion layer reads

Table 1: Relative improvement (%) over pickup-distance-based KM matching in the three cities over the full day.

Method	City A		City B		City C	
	GMV	CR	GMV	CR	GMV	CR
TVal	+2.24	+2.14	+1.87	+1.63	+2.56	+2.48
GRC	+0.73	−3.42	+1.15	−2.18	+0.41	−1.87
Llama-3.3-70B	+2.34	+2.76	+1.92	+2.31	+2.68	+3.12
Qwen3-Next-80B	+2.41	+2.54	+2.08	+2.12	+2.75	+2.89
DeepSeek-R1	+2.53	+4.57	+2.17	+3.89	+2.91	+4.93
Kimi-K2	+1.96	+4.77	+1.63	+4.05	+2.24	+5.18
GPT-OSS-120B	+2.44	+5.75	+2.06	+5.12	+2.79	+6.08
Gemini-3-Flash	+1.41	+4.62	+1.08	+3.94	+1.72	+4.95
Gemini-3.1-Pro	+2.95	+5.48	+2.51	+4.83	+3.28	+5.81
ProfiLLM-DPO	**+4.02**	+6.03	**+3.58**	+5.47	**+4.35**	+6.41
ProfiLLM‡	+3.52	**+7.10**	+3.14	**+6.52**	+3.87	**+7.53**

the embedding $\mathbf{e}_u$ of the relevant side, that is, $\mathbf{e}_d$ for acceptance and driver cancellation and $\mathbf{e}_p$ for passenger cancellation, together with a few order-context features $\mathbf{x}'_c$ and ℓ_c^{base}, and outputs

$$\ell_c = \ell_c^{\text{base}} + m_u\, \sigma(\gamma_c)\, (\ell_c^{\text{prof}} - \ell_c^{\text{base}}), \tag{8}$$

where ℓ_c^{prof} is a profile-conditioned logit computed from $(\mathbf{e}_u, \mathbf{x}'_c)$ by a two-token Transformer, γ_c is a learned gate, and $m_u = 0$ for users with the default embedding. The gate is an input-dependent counterpart of λ in Eq. (4), and users without a profile receive exactly the production prediction. The fusion layer is trained on the training window with the predictor frozen. The predicted outcomes are then aggregated into a matching weight w_c quantifying the expected utility of each assignment. The regional priors add a bounded bonus to w_c, computed from $\mathbf{r}_{g,s}$ of the pickup grid in the current slot and of the destination grid in the expected arrival slot; it is higher where pickup demand exceeds supply and where follow-up demand and the value of the destination are high. We applied $\mathcal{R}$ in the simulator but not in the online test. Finally, the optimal matching $\mathcal{M}^*$ that maximizes $\sum_{c\in\mathcal{M}} w_c$ is computed via the Kuhn-Munkres algorithm, completing one dispatching cycle.

Users unseen in training receive $\mathbf{e}_0$ and hence the production prediction until a daily reassignment gives them a cluster.

3.5 Refresh and Drift

Refresh is planned at three levels, each including the cheaper ones: daily *user reassignment* to the nearest centroid, which takes under 30 s for City A and gives new users a cluster once they have history; weekly *profile regeneration* with one DPO call per cluster and a retrained fusion layer, also when per-cluster calibration or AUC degrades; and monthly *re-mining and re-clustering* of $\mathcal{K}$, $\mathcal{R}$, and $\mathcal{A}$, also when cluster sizes, the shares of users on $\mathbf{e}_0$ or far from all centroids, or city-level conditions such as pricing shift. The online test ran without refresh, yet the treatment's completion rate exceeded the control's on all 14 days, including Carnival; how fast the gain decays without refresh and how to refresh across seasons remain open. Our online appendix gives each level's end-to-end time and cost on the 44.3M City A records and the serving latency.

4 EXPERIMENTS

We conduct extensive experiments on real-world industrial datasets from DiDi's ride-hailing platform to evaluate ProfiLLM [1].

[1] https://ProfiLLM.github.io

4.1 Experimental Setup

4.1.1 Datasets. We evaluate ProfiLLM on real-world order dispatching from three Brazilian cities spanning distinct supply–demand regimes: **City A**, medium-scale and supply-constrained with the highest driver utilization; **City B**, medium-scale and supply-relaxed with moderate order density; and **City C**, large-scale and high-demand with complex traffic. For each city, we use 38 days of historical data for training, including global knowledge mining and profile exploration, and five days for testing. In City A, the 44.3M-record analysis log covers 333,166 passengers with at least one order and 12,128 drivers with at least ten broadcasts, to which the long-tail and heterogeneity statistics apply; the cluster registry contains all 348,464 users, 334,657 passengers and 13,807 drivers.

4.1.2 Evaluation Metrics. We evaluate performance at two levels. *(i) Dispatching level.* We report **GMV** (Gross Merchandise Value) together with six realized rates computed in the dispatching simulator or the online A/B test: **CR** (Completion Rate), **DAR** (Driver Acceptance Rate, per broadcast), **DCR** (Driver Cancellation Rate, post-acceptance driver cancel, including reassigned orders), **PCR** (Passenger Cancellation Rate, post-acceptance passenger cancel), **CBA** (Cancel Before Accept, share of orders the passenger cancels before any driver accepts), and **BER** (Bad Experience Rate, share of completed orders with excessively long pickup distance). *(ii) Prediction level.* For each candidate OD pair, the prediction model produces a probability for each of four behavioral events. We report **AUC** for each task, denoted **Accept**, **D-Cancel**, **P-Cancel**, and **Success**, in one-to-one correspondence with DAR, DCR, PCR, and CR. The two views measure the same four events at different stages of the pipeline and can therefore move differently when a system change affects matching weights and per-pair prediction to different degrees. CBA and BER are pure dispatching-level rates without a direct prediction-task counterpart, since CBA occurs before any OD-pair assignment is made and BER reflects realized pickup quality after matching.

4.1.3 Baselines. We compare ProfiLLM against two categories of baselines. **Traditional dispatching methods** rely on structured features without user profiling: TVal [32] and GRC [34]. **LLM-based profiling methods** use the same clustering and knowledge mining pipeline as ProfiLLM but differ in the LLM backbone for profile generation, including Llama-3.3-70B [4], Qwen3-Next-80B [33], DeepSeek-R1 [6], Kimi-K2 [26], Gemini-3-Flash [25], Gemini-3.1-Pro [25], and GPT-OSS-120B, each with one single-pass profile per cluster. **ProfiLLM** with exploration-selected profiles but without DPO fine-tuning, and **ProfiLLM-DPO** with the full framework including DPO-aligned profile generation. **Structured-only baseline.** The reference is the mature production predictor with about 500 features, including minute-level supply–demand counts, live-traffic ETAs, weather, and about 230 per-user aggregates over 1- to 120-day and lifetime windows.

4.1.4 Implementation Details. Knowledge mining runs on Gemini-3-Pro and exploration on GPT-OSS-120B, and the DPO-aligned generator is Qwen3-8B. In City A, the agent chose 38 driver and 58 passenger clusters, whose profiles are used in Table 2 and online. Our online appendix gives all settings and a granularity study with 8–256 clusters per side and nine generators, which places this choice

Table 2: Multi-task prediction AUC improvement (%) over Structured Only baseline. Higher is better.

Method	City A				City B				City C			
	Accept	D-Cancel	P-Cancel	Success	Accept	D-Cancel	P-Cancel	Success	Accept	D-Cancel	P-Cancel	Success
Llama-3.3-70B	-1.10	-0.71	+0.19	-1.14	-0.64	+0.38	-0.38	-0.45	-0.01	-0.34	+0.25	+0.01
Qwen3-Next-80B	-0.22	-0.38	+1.65	+0.02	-0.52	-0.40	-5.71	-7.57	-0.03	-0.16	+0.27	-0.06
DeepSeek-R1	+0.06	+0.23	+2.05	+0.25	+0.31	+1.85	+1.06	+0.48	+0.21	-0.13	+0.04	+0.14
Kimi-K2	-0.17	+0.82	+2.11	-0.07	-2.44	-0.44	-6.33	-1.91	+0.50	-0.11	+0.40	+0.45
Gemini-3-Flash	+0.10	+0.53	+1.83	+0.42	+0.24	+1.76	-0.11	+0.38	+0.03	-0.26	+0.37	+0.04
Gemini-3.1-Pro	-0.08	-0.68	+2.37	+0.56	-0.44	+0.50	+0.24	-0.31	+0.02	-0.03	+0.10	+0.05
GPT-OSS-120B	-0.02	+0.14	+1.83	+0.17	+0.11	+1.64	+0.63	+0.29	-0.09	-0.02	+0.44	-0.06
ProfiLLM-DPO	+1.51	+2.76	+6.02	+1.72	+2.25	+4.98	+5.55	+2.58	+0.65	+5.93	+5.30	+2.37
ProfiLLM	+1.56	+3.88	+6.14	+1.80	+2.26	+4.98	+6.00	+2.60	+0.84	+5.95	+5.65	+2.48

on a plateau: beyond 16 clusters per side, ProfiLLM-DPO's AUC varies by only 0.21–0.32 points per task, while cost grows linearly.

4.2 Overall Performance

Table 1 presents the simulator-based evaluation results across three cities. The simulator replays one day outside the training window per city with all real orders and drivers in production KM cycles. All methods share one outcome model, ProfiLLM's own predictor, which Table 1 marks with ‡, and only ProfiLLM applies $\mathcal{R}$, so we read the simulator for the direction of effects only; our online appendix gives its protocol, limitations, and time-of-day results. From the results, we make the following observations. (1) ProfiLLM variants consistently outperform all baselines. Both ProfiLLM and ProfiLLM-DPO achieve the highest GMV and CR among all methods. Notably, the two variants exhibit complementary strengths: ProfiLLM-DPO achieves the best GMV, up to +4.35% in City C, while ProfiLLM attains the best CR, up to +7.53% in City C. ProfiLLM's CR advantage is expected, however, since the shared outcome model is its own predictor; ProfiLLM-DPO, a different checkpoint, still has the highest GMV in every city. (2) LLM-based profiles substantially outperform traditional methods. TVal and GRC, which rely solely on structured numerical features, achieve moderate GMV gains but inconsistent CR, with GRC negative in all three cities. In contrast, all LLM-based methods deliver positive improvements on both metrics, suggesting that semantic user profiles capture behavioral patterns that handcrafted features expose only partially. (3) Utility alignment matters more than model scale in the simulator. Among LLM-based methods, larger backbones generally yield better profiles, yet our DPO-aligned Qwen3-8B consistently outperforms much larger models such as Gemini-3.1-Pro without alignment. This suggests that optimizing for downstream prediction utility is more effective than scaling the LLM backbone alone. The like-for-like margins are small, however: ProfiLLM-DPO exceeds Gemini-3.1-Pro by +1.04% GMV in every city and TVal by about +1.7%, far below its +4.02% gain over KM-Pickup in City A; and with the gated fusion layer, DPO raises the prediction AUC of Qwen3-8B by 0.22–0.96% in every city–task cell.

4.3 Prediction Performance

Table 2 reports the AUC improvement of the multi-task predictor trained with each method's profiles over the structured-only baseline. The results reveal that naively applying LLMs for profile generation does not guarantee downstream utility. While some baseline LLMs achieve modest positive improvements, others actually degrade prediction performance: Kimi-K2 drops P-Cancel AUC by 6.33% in City B, and Qwen3-Next-80B yields a 7.57% decrease in Success. In contrast, both ProfiLLM and ProfiLLM-DPO deliver consistent positive gains across all tasks and cities, with ProfiLLM achieving up to +6.14% on P-Cancel in City A, +5.95% on D-Cancel in City C, and +2.60% on Success in City B. The baseline rows, however, use the naive integration of Challenge 3. Re-evaluated with ProfiLLM's gated fusion layer, which was trained with ProfiLLM's embeddings and if anything favors ProfiLLM, every backbone improves all four tasks in all cities, by +0.51% to +6.07%, and ProfiLLM remains best in all 12 city–task cells, two of them tied, but by at most +1.02% for D-Cancel in City A and +0.28% elsewhere. The largest improvements are observed for cancellation prediction, which aligns with our pilot study: cancellation behaviors depend heavily on contextual preferences such as sensitivity to pickup distance that handcrafted features struggle to capture. Finally, ProfiLLM and ProfiLLM-DPO achieve comparable prediction AUC, with ProfiLLM slightly ahead on most metrics; DPO lifts the untuned Qwen3-8B by 0.22–0.96% in every cell, to the level of much larger backbones. Part of the gain over the production predictor may stem from the fusion layer itself, which a fusion-only control, left to future work, would separate.

4.4 Ablation Study

Figure 4 reports results on City A as relative change compared to the full ProfiLLM. Because the variants come from our earlier retrained-predictor pipeline, the bars combine the effect of each component with that of the integration. The re-evaluation of Section 4.3 with the gated fusion layer gives two same-pipeline ablations: *w/o exploration*, one single-pass GPT-OSS-120B profile per cluster, lowers AUC by up to 1.04% for D-Cancel in City A and by at most 0.28% elsewhere, and *w/o DPO*, the un-tuned Qwen3-8B, lowers AUC by 0.22–0.95% in every cell. Relative to the structured-only predictor in the earlier pipeline, profiles without global knowledge lower Accept, D-Cancel, and Success AUC by 1.77–2.48%, and per-user profiles lower P-Cancel AUC by 9.08%. (1) **w/o Global Knowledge:** Removing global knowledge causes substantial drops across all tasks, such as −6.12% for D-Cancel, confirming that platform-level knowledge provides essential grounding for generating predictive profiles. (2) **w/o User Clustering:** Replacing cluster-level profiles with per-user profiles yields the largest degradation, −14.33% for P-Cancel, as 96% of passengers lack sufficient history for reliable individual profiling. (3) **w/o Exploration:** Skipping iterative refinement lowers GMV by 1.32% and also degrades prediction, confirming that the explore-evaluate-refine loop is necessary for utility alignment. (4) **w/o DPO Fine-tuning:** We replace the DPO-aligned generator

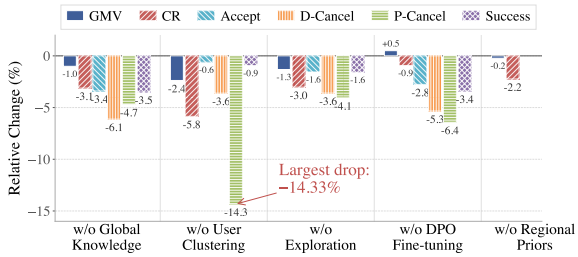


Figure 4: Ablation on City A. Regional Priors only affect matching weights, hence no prediction-AUC bar.

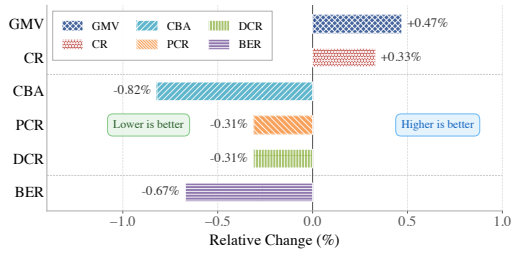


Figure 5: Online A/B over 14 days in City A: treatment-vs.-control relative change. The GMV and CR gains are significant ($p = 0.03$ and 0.037); the four decreases are not ($p \geq 0.07$).

with the un-finetuned base Qwen3-8B in a single-pass setting. Prediction AUC drops substantially, by 6.42% for P-Cancel; with the gated fusion layer the drop is 0.93%, so most of it reflects the earlier integration. (5) **w/o Regional Priors:** Lowers CR by 2.24% without affecting prediction, consistent with priors being incorporated into matching weights rather than the prediction model.

4.5 Production Deployment

We deployed ProfiLLM in DiDi's production environment and report results from a 14-day A/B test in City A. The test is a switchback [2] on the city-center traffic of City A: control and treatment alternated every 30 minutes from February 5 to 18, 2026 (336 slices per arm), and DiDi's A/B platform reports a two-sided 95% confidence interval (CI) and p-value per metric. As shown in Figure 5, GMV and CR rise together and significantly, by 0.47% (95% CI [+0.04%, +0.90%], $p = 0.03$) and 0.33% ([+0.02%, +0.64%], $p = 0.037$), suggesting better matches rather than a trade of completion for revenue. CBA, PCR, DCR, and BER all fall, but not significantly: the CI of CBA is [−1.72%, +0.08%] ($p = 0.07$), and those of the others span ±1.3% to ±2.1% ($p \geq 0.53$). The per-broadcast DAR fell by 0.56% ([−1.53%, +0.41%], $p = 0.26$) only because each order was broadcast 0.56% more often, and no metric worsened significantly.

These online deltas are several times smaller than the simulation gains of Section 4.2 (+0.47% against +3.52% GMV in City A for the deployed ProfiLLM), but the two are not comparable: (i) simulated gains are relative to KM-Pickup, whereas the online control already uses the production predictor, and ProfiLLM's simulated margin over Gemini-3.1-Pro and TVal is only +0.55% to +1.28%; (ii) even these margins are overstated, since the simulator's outcome model is ProfiLLM's own predictor; (iii) online, $\mathcal{R}$ is not used and only users with a profile are affected; and (iv) drivers' positions and states carry over between adjacent slices, which tends to attenuate measured effects [2]. The online CI overlaps the lower end of these margins, as (ii)–(iv) predict; it establishes the direction of the gain, but its size across cities and seasons needs a longer, multi-city test.

5 RELATED WORK

Ride-hailing Order Dispatching. Order dispatching has evolved from proximity-based greedy matching [11, 27, 36, 37] to RL methods capturing long-term dynamics: demand–supply forecasting with combinatorial optimization [32], mean-field multi-agent RL [18], and cooperative Markov games [34]. Prediction-side work models driver acceptance [29] and cancellations [3]. These rely on structured numerical features, leaving semantic factors unexploited; our LLM-generated profiles capture them.

LLM Data Pipelines for Applied ML. Closest to our setting, LEADRE [12] deploys DPO-aligned LLMs in Tencent's display-advertising pipeline, and SiriusBI [8] productionizes multi-round NL2SQL for enterprise BI. On the agentic side, DocETL [22], AutoPrep [5], SQL-Factory [13], LLM-AutoDP [7], and DBAIOps [41] rewrite or compose LLM-agent operators for document, tabular, SQL, and database tasks; KATS [30] pairs an offline LLM knowledge graph with online retrieval. Unlike these, ProfiLLM is the first to bring an agentic LLM data pipeline into a real-time dispatcher.

LLM-based Agents and User Profiling. LLM agents such as ReAct [35], Toolformer [20], and Data-Copilot [39] extend their capabilities via tools and have been applied to traffic analysis [16, 38], though mainly for general insights rather than production prediction. We instead design a tool-augmented mining workflow that yields actionable knowledge directly improving prediction.

Separately, user behavior modeling spans sequential models [9, 23], deep interest networks [40], and recent LLM-based profiling [14, 28, 31]; in ride-hailing, prior work estimates driver value under behavioral heterogeneity [24] and models passenger demand [10]. We do not compare with sequential models, which predict next items rather than order–driver outcomes; adding a sequence encoder to the outcome predictor is future work. RLHF [1, 17] aligns LLMs via reward modeling and PPO [21] but is complex to train; DPO [19] optimizes directly from preference pairs and has many variants. We contribute a novel application: constructing preference pairs from task utility rather than human judgment to train a compact generator for cheap profile refresh.

6 CONCLUSION

We presented ProfiLLM, a practical framework that bridges LLM-based semantic profiling with real-time ride-hailing order matching. ProfiLLM addresses three key challenges through two synergistic modules: *Tool-Augmented Global Knowledge Mining*, which equips an LLM agent with 27 analytical tools to extract global knowledge, clustering rules, and regional priors; and *Utility-Aligned Profile Exploration*, which iteratively refines cluster-level profiles via a lightweight prediction proxy and DPO fine-tuning. Experiments on DiDi's platform across three cities demonstrate up to +6.14% relative AUC improvement and +4.35% GMV gain in simulation, with a 14-day production A/B test confirming gains of +0.47% GMV and +0.33% completion rate. Future work will isolate the effect of the profile text with a fusion-only control and add a sequence encoder to the outcome predictor. We also plan longer online tests in more cities, with periodic profile refresh.

REFERENCES

[1] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).

[2] Iavor Bojinov, David Simchi-Levi, and Jinglong Zhao. 2023. Design and Analysis of Switchback Experiments. *Management Science* 69, 7 (2023), 3759–3777.

[3] M Keith Chen and Michael Sheldon. 2016. Dynamic pricing in a labor market: Surge pricing and flexible work on the Uber platform. *Ec* 16 (2016), 455.

[4] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[5] Meihao Fan, Ju Fan, Nan Tang, Lei Cao, Guoliang Li, and Xiaoyong Du. 2025. AutoPrep: Natural Language Question-Aware Data Preparation with a Multi-Agent Framework. *Proc. VLDB Endow.* 18, 10 (2025), 3504–3517. https://doi.org/10.14778/3748191.3748211

[6] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[7] Wei Huang, Anda Cheng, Yinggui Wang, Lei Wang, and Tao Wei. 2026. LLM-AutoDP: Automatic Data Processing via LLM Agents for Model Fine-tuning. *Proc. VLDB Endow.* 19, 5 (2026), 794–807. https://doi.org/10.14778/3796195.3796196

[8] Jie Jiang, Haining Xie, Siqi Shen, Yu Shen, Zihan Zhang, Meng Lei, Yifeng Zheng, Yang Li, Chunyou Li, Danqing Huang, Yinjun Wu, Wentao Zhang, Bin Cui, and Peng Chen. 2025. SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence. *Proc. VLDB Endow.* 18, 12 (2025), 4860–4873. https://doi.org/10.14778/3750601.3750610

[9] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[10] Jintao Ke, Feng Xiao, Hai Yang, and Jieping Ye. 2020. Learning to delay in ride-sourcing systems: A multi-agent deep reinforcement learning framework. *IEEE Transactions on Knowledge and Data Engineering* 34, 5 (2020), 2280–2292.

[11] Der-Horng Lee, Hao Wang, Ruey Long Cheu, and Siew Hoon Teo. 2004. Taxi dispatch system based on current demands and real-time traffic conditions. *Transportation Research Record* 1882, 1 (2004), 193–200.

[12] Fengxin Li, Yi Li, Yue Liu, Chao Zhou, Yuan Wang, Xiaoxiang Deng, Wei Xue, Dapeng Liu, Lei Xiao, Haijie Gu, Jie Jiang, Hongyan Liu, Biao Qin, and Jun He. 2025. LEADRE: Multi-Faceted Knowledge Enhanced LLM Empowered Display Advertisement Recommender System. *Proc. VLDB Endow.* 18, 12 (2025), 4763–4776. https://doi.org/10.14778/3750601.3750602

[13] Jiahui Li, Tongwang Wu, Yuren Mao, Yunjun Gao, Yajie Feng, and Huaizhong Liu. 2025. SQL-Factory: A Multi-Agent Framework for High-Quality and Large-Scale SQL Generation. *Proc. VLDB Endow.* 19, 3 (2025), 292–305. https://doi.org/10.14778/3778092.3778093

[14] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2024. Once: Boosting content-based recommendation with both open-and closed-source large language models. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 452–461.

[15] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605.

[16] Yansong Ning, Shuowei Cai, Wei Li, Jun Fang, Naiqiang Tan, Hua Chai, and Hao Liu. 2025. Dima: An llm-powered ride-hailing assistant at didi. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4728–4739.

[17] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[18] Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. 2020. Ride-hailing order dispatching at didi via reinforcement learning. *INFORMS Journal on Applied Analytics* 50, 5 (2020), 272–286.

[19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.

[20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.

[21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[22] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3035–3048. https://doi.org/10.14778/3746405.3746426

[23] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

[24] Xiaocheng Tang, Fan Zhang, Zhiwei Qin, Yansheng Wang, Dingyuan Shi, Bingchen Song, Yongxin Tong, Hongtu Zhu, and Jieping Ye. 2021. Value function is all you need: A unified learning framework for ride hailing platforms. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3605–3615.

[25] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[26] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. 2025. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534* (2025).

[27] Jing-Peng Wang, Hai Wang, Peng Liu, and Hai-Jun Huang. 2025. Order dispatching strategy and pricing scheme in ride-sourcing markets with consideration of service cancellation. *Transportation Research Part B: Methodological* 199 (2025), 103266.

[28] Lu Wang, Di Zhang, Fangkai Yang, Pu Zhao, Jianfeng Liu, Yuefeng Zhan, Hao Sun, Qingwei Lin, Weiwei Deng, Dongmei Zhang, et al. 2025. Lettingo: Explore user profile generation for recommendation system. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 2985–2995.

[29] Yansheng Wang, Yongxin Tong, Cheng Long, Pan Xu, Ke Xu, and Weifeng Lv. 2019. Adaptive dynamic bipartite graph matching: A reinforcement learning approach. In *2019 IEEE 35th international conference on data engineering (ICDE)*. IEEE, 1478–1489.

[30] Zixin Wei, Yucan Guo, Jinyang Li, Xiaolin Han, Xiaolong Jin, and Chenhao Ma. 2026. Revisiting Task-Oriented Dataset Search in the Era of Large Language Models: Challenges, Benchmark, and Solution. *Proc. VLDB Endow.* 19, 5 (2026), 973–986. https://doi.org/10.14778/3796195.3796209

[31] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 12–22.

[32] Zhe Xu, Zhixin Li, Qingwen Guan, Dingshui Zhang, Qiang Li, Junxiao Nan, Chunyang Liu, Wei Bian, and Jieping Ye. 2018. Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 905–913.

[33] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[34] Zhaoxing Yang, Haiming Jin, Guiyun Fan, Min Lu, Yiran Liu, Xinlang Yue, Hao Pan, Zhe Xu, Guobin Wu, Qun Li, et al. 2024. Rethinking order dispatching in online ride-hailing platforms. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3863–3873.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

[36] Xinlang Yue, Yiran Liu, Fangzhou Shi, Sihong Luo, Chen Zhong, Min Lu, and Zhe Xu. 2024. An End-to-End Reinforcement Learning Based Approach for Micro-View Order-Dispatching in Ride-Hailing. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 5054–5061.

[37] Lingyu Zhang, Tao Hu, Yue Min, Guobin Wu, Junying Zhang, Pengcheng Feng, Pinghua Gong, and Jieping Ye. 2017. A taxi order dispatch model based on combinatorial optimization. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2151–2159.

[38] Siyao Zhang, Daocheng Fu, Wenzhe Liang, Zhao Zhang, Bin Yu, Pinlong Cai, and Baozhen Yao. 2024. Trafficgpt: Viewing, processing and interacting with traffic foundation models. *Transport Policy* 150 (2024), 95–105.

[39] Wenqi Zhang, Yongliang Shen, Weiming Lu, and Yueting Zhuang. 2023. Data-copilot: Bridging billions of data and humans with autonomous workflow. *arXiv preprint arXiv:2306.07209* (2023).

[40] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.

[41] Wei Zhou, Peng Sun, Xuanhe Zhou, Qianglei Zang, Ji Xu, Tieying Zhang, Guoliang Li, and Fan Wu. 2026. DBAIOps: A Reasoning LLM-Enhanced Database Operation and Maintenance System using Knowledge Graphs. *Proc. VLDB Endow.* 19, 6 (2026), 1319–1331. https://doi.org/10.14778/3797919.3797937